codeHive

# Counting Sort

The Non-Comparison Frequency Algorithm

Module 11 | Advanced Sorting Series

# 1. Beyond Comparisons

**Counting Sort** is a unique algorithm because it does **not** compare values against each other. Instead, it sorts by counting the frequency of each distinct value.

While algorithms like Quicksort can sort any data type, Counting Sort is specialized: it only works with **non-negative integers** in a limited range.

## The Logic Chain

1. Identify the **Maximum** value in the input array.
2. Create a **Counting Array** of size `Max + 1`.
3. Traverse the input array and increment counts at the index equal to the value.
4. Rebuild the sorted array using the frequencies stored.

# 2. Essential Conditions

Before choosing Counting Sort, you must verify these three constraints:

## A. Integer Values

The algorithm relies on mapping values directly to array indices. This only works for whole numbers.

## B. Non-Negative Values

Standard implementations cannot handle negative numbers because a negative value like -5 cannot be an index in the counting array.

## C. Limited Range (K)

If you have only 5 numbers to sort, but one of them is 1,000,000, you would need a massive counting array for very little data. This makes the algorithm highly inefficient.

# 3. Manual Run-Through

Let's sort: `[ 2, 3, 0, 2, 3, 2 ]`

**Step 1: The Counting Array**

We find the max value is 3. We create a counter for [0, 1, 2, 3].


**Step 2: Rebuilding**

We see Val 0 appears once, Val 2 appears three times, and Val 3 appears twice.

Output: `[ 0, 2, 2, 2, 3, 3 ]`

# 4. Python Implementation

```
def countingSort(arr):
    if not arr:
        return arr

    # Find the range
    max_val = max(arr)
    count = [0] * (max_val + 1)

    # Step 1: Count occurrences
    for num in arr:
        count[num] += 1

    # Step 2: Clear and rebuild
    arr[:] = []

    for num, freq in enumerate(count):
        # Add the 'num' back 'freq' number of times
        arr.extend([num] * freq)

    return arr

# Testing
data = [4, 2, 2, 6, 3, 3, 1, 6, 5]
print("Sorted:", countingSort(data))
```

# 5. Efficiency Analysis

Counting Sort doesn't fit the standard comparison-based logic. Its speed depends on both **n** (number of elements) and **k** (range of values).

TIME COMPLEXITY

$$O(n + k)$$

**Best Case: O(n)**

When k is small compared to n.

**Worst Case: O(k)**

When k is massive compared to n.

**The Takeaway:** Counting Sort is "Linear" when you are sorting many elements that fall within a tight range of values. It is exceptionally fast for things like sorting test scores (0-100) or RGB color values (0-255).

# Test Your Logic

**Exercise:** Using Counting Sort on the array:

`[1, 0, 5, 3, 3, 1, 3, 3, 4, 4]`

How would the counting array look for values 0 through 5?

Answer: [1, 2, 0, 4, 2, 1]